

cUtils

A Generic, Unified Data Structures Library in C

Linked List · Queue · Stack · Tree · Graph

August 3, 2026

A single header/implementation pair providing one generic node shape that backs a doubly linked list, a queue, a stack, and a unified tree/graph structure — all supporting `int`, `float`, `char`, and `string` values through one tagged union, plus a from-scratch merge sort and an ASCII-art visualizer that draws every structure the way it is drawn on paper.

Contents

1	Overview	3
1.1	Project layout	3
1.2	Build	3
2	The Generic Value System	4
2.1	Value builders	4
2.2	TYPE_PTR: the type that ties everything together	4
3	Doubly Linked List	5
3.1	Unified insertion	5
3.2	Pop: delete <i>and</i> return	5
3.3	Full API	5
4	Sorting: In-Place Merge Sort	6
5	Queue and Stack	7
5.1	Queue (FIFO)	7
5.2	Stack (LIFO)	7
6	Unified Tree / Graph Structure	8
6.1	GNodeArray	8
6.2	The registry pattern	8
6.3	Tree mode	9
6.3.1	Traversals	9
6.4	Graph mode	9
7	ASCII-Art Visualization	10
7.1	Linked list, queue, stack: box diagrams	10
7.2	Tree: a recursive 2D layout	10
7.3	Graph: circular layout with a hand-drawn line style	10
8	Testing	12
8.1	Aligned, wrapped terminal logging	12

8.2	Test structure	12
9	Appendix: Full API Reference	13
10	Appendix: Complexity Summary	14

1 Overview

`cUtils` is a two-file C library (`cUtils.h` / `cUtils.c`) built up incrementally from a simple integer-only doubly linked list into a single, generic, reusable toolkit covering five classic data structures:

- A **doubly linked list** with unified insertion/deletion and an in-place **merge sort**.
- A **queue** (FIFO) and a **stack** (LIFO), both built directly on top of the linked list.
- A **unified tree/graph** structure: the same node type and the same adjacency-list mechanism serve both an N-ary tree and a general graph.

Every one of these structures stores values through the same tagged-union system, so a single `LLnode` or `GNode` can hold an `int`, a `float`, a `char`, a heap-owned string, or a raw pointer, without any code duplication between types.

C has no generics or templates. The two idiomatic ways to fake them are (a) `void*` with manual casting, or (b) a tagged union: an `enum` that records which type is stored, paired with a `union` that holds the value. This project uses the tagged-union approach throughout, since it keeps values inline (no extra heap allocation for primitives) and lets every print/compare/sort routine `switch` on one field to know what it's looking at.

1.1 Project layout

File	Contents
<code>cUtils.h</code>	Type definitions and public function prototypes
<code>cUtils.c</code>	Implementation of every function declared in the header
<code>test.c</code>	A self-contained test suite covering every public function

1.2 Build

```
gcc -Wall -Wextra -std=c11 -Iinclude -o bin/test tests/test.c
    include/cUtils.c
./bin/test
```

No external libraries are required — not even `-lm`. The graph visualizer needs `sin/cos` for its circular layout, so this project ships its own small Taylor-series approximation (`art_sin` / `art_cos`) instead of linking against `libm`, keeping the build a plain two-file `gcc` command on any platform.

2 The Generic Value System

Every structure in the library is built on the same two types:

```

1 typedef enum { TYPE_INT, TYPE_FLOAT, TYPE_CHAR, TYPE_STR, TYPE_PTR }
   DataType;
2
3 typedef union {
4     int    i_val;
5     float  f_val;
6     char   c_val;
7     char   *s_val;    // heap-allocated copy, owned by whichever node
                        // holds it
8     void   *p_val;    // raw address -- used for tree/graph adjacency
                        // lists
9 } Data;
```

Every node carries a `DataType` tag alongside its `Data` union, so any function that needs to read, print, compare, or free the value can `switch` on the tag to pick the right union member.

2.1 Value builders

Rather than filling in the union by hand at every call site, five small helper functions build a `Data` value from a plain C value:

Function	Purpose
<code>make_int(int v)</code>	wraps an <code>int</code>
<code>make_float(float v)</code>	wraps a <code>float</code>
<code>make_char(char v)</code>	wraps a <code>char</code>
<code>make_str(const char *v)</code>	heap-copies a string (<code>strdup</code> -style)
<code>make_ptr(void *p)</code>	wraps a raw pointer

`make_str` always allocates its own copy of the string, so the caller never has to worry about the lifetime of the original buffer — the list node owns the copy and frees it when the node is deleted or the list is freed.

2.2 TYPE_PTR: the type that ties everything together

`TYPE_PTR` does not represent a value the end user stores directly. It is the mechanism that lets the *same* `LLnode` list used for the linked list / queue / stack also serve as the adjacency list for the tree and graph structures: a `GNode`'s `neighbors` field is itself an `LLnode*` list, where every entry is a `TYPE_PTR` value holding the address of another `GNode`.

3 Doubly Linked List

```

1 typedef struct llnode {
2     Data data;
3     DataType type;
4     struct llnode *right;    // next
5     struct llnode *left;    // previous
6 } LLnode;

```

3.1 Unified insertion

The three classic insertion operations (insert at beginning, insert at end, insert at position) are collapsed into a single function:

```

1 #define LL_END (-1)
2 void ll_add(LLnode **head_ref, DataType type, Data value, int pos);

```

Call	Behaves like
ll_add(h, t, v, LL_END)	append (old ll_add_end)
ll_add(h, t, v, 0)	prepend (old ll_add_beg)
ll_add(h, t, v, pos)	insert before index pos (old ll_add_at)

C has no default/optional arguments. `LL_END` (`-1`) is a sentinel value meaning “no position given, so append” — the standard idiomatic way to fake an optional parameter in C.

3.2 Pop: delete *and* return

Unlike a plain delete, both pop functions remove the node *and* hand its value back to the caller:

```

1 Data ll_pop_beg(LLnode **head_ref, DataType *type_out);
2 Data ll_pop_end(LLnode **head_ref, DataType *type_out);

```

`type_out` is an optional out-parameter telling the caller which member of the returned `Data` union is valid. Popping a `TYPE_STR` value transfers ownership of the heap string to the caller, who is then responsible for calling `free()` on it.

3.3 Full API

Function	Description
<code>create_node(type, data)</code>	allocate one node
<code>ll_len(head_ref)</code>	length of the list
<code>ll_add(head_ref, type, value, pos)</code>	unified insert (see above)
<code>ll_pop_beg(head_ref, type_out)</code>	remove & return the front value
<code>ll_pop_end(head_ref, type_out)</code>	remove & return the back value
<code>ll_del_at(head_ref, pos)</code>	delete at <code>pos</code> without returning the value
<code>ll_peek(head_ref)</code>	draw the list as a box diagram
<code>ll_free_list(head_ref)</code>	free every node (and owned strings)
<code>ll_sort(head_ref)</code>	ascending merge sort

4 Sorting: In-Place Merge Sort

`ll_sort` sorts the list ascending using a textbook doubly-linked-list merge sort, built from three pieces:

1. `ll_split` — splits a list of ≥ 2 nodes roughly in half using the classic slow/fast pointer technique (`fast` advances two nodes per step, `slow` advances one; when `fast` runs out, `slow` sits at the midpoint).
2. `ll_merge` — merges two already-sorted sublists into one, recursively, restoring both `left` and `right` links as it goes.
3. `ll_merge_sort_rec` — the classic recursive skeleton: split, sort each half, merge.

```

1 static LLnode *ll_merge_sort_rec(LLnode *head, DataType type) {
2     if (head == NULL || head->right == NULL) {
3         return head; // 0 or 1 node -- already
                        sorted
4     }
5     LLnode *second_half = ll_split(head);
6     LLnode *left_sorted = ll_merge_sort_rec(head, type);
7     LLnode *right_sorted = ll_merge_sort_rec(second_half, type);
8     return ll_merge(left_sorted, right_sorted, type);
9 }

```

Comparisons go through a small type-aware helper:

```

1 static int data_compare(Data a, Data b, DataType type) {
2     switch (type) {
3         case TYPE_INT:    return (a.i_val > b.i_val) - (a.i_val < b.
                           i_val);
4         case TYPE_FLOAT: return (a.f_val > b.f_val) - (a.f_val < b.
                           f_val);
5         case TYPE_CHAR:  return (a.c_val > b.c_val) - (a.c_val < b.
                           c_val);
6         case TYPE_STR:   return strcmp(a.s_val, b.s_val);
7         case TYPE_PTR:   return (a.p_val > b.p_val) - (a.p_val < b.
                           p_val);
8     }
9     return 0;
10 }

```

Time: $O(n \log n)$ — the list is halved $\log n$ times, and each level does $O(n)$ work merging.
Space: $O(\log n)$ auxiliary (recursion stack only; the sort is done in place on the existing nodes, no extra list is allocated).

`ll_sort` assumes the list is *homogeneous* — every node the same `DataType`. Sorting mixed types together isn't well defined, so the comparison type is taken from the first node.

5 Queue and Stack

Both are thin wrappers around the same `LLnode` list, differing only in which end they pop from.

5.1 Queue (FIFO)

```
1 void enqueue(LLnode **head_ref, DataType type, Data value);    //  
    ll_add(..., LL_END)  
2 Data dequeue(LLnode **head_ref, DataType *type_out);          //  
    ll_pop_beg(...)  
3 void display_queue(LLnode **head_ref);                          //  
    FRONT/REAR box diagram
```

5.2 Stack (LIFO)

```
1 void push(LLnode **head_ref, DataType type, Data value);      //  
    ll_add(..., LL_END)  
2 Data pop(LLnode **head_ref, DataType *type_out);              //  
    ll_pop_end(...)  
3 void display_stack(LLnode **head_ref);                         //  
    TOP box diagram
```

Both `enqueue` and `push` insert at the same end (`LL_END`); the only difference between a queue and a stack is which end *removal* happens from — the front for a queue, the back for a stack. This is why both are only a few lines of code on top of the same list.

Every queue/stack operation is $O(1)$: `ll_add(..., LL_END)` tracks the list's own `left/right` links rather than re-walking the list (the head's back-link and each node's forward link are always kept consistent), and `ll_pop_beg/ll_pop_end` touch only the very first or very last node.

6 Unified Tree / Graph Structure

A tree is just a graph where every node (except the root) has exactly one parent and there are no cycles. This project takes that observation literally: trees and graphs share one node type, `GNode`, and one adjacency-list mechanism (an `LLnode` list of `TYPE_PTR` entries).

- In **tree mode**, `neighbors` holds a node's children.
- In **graph mode**, `neighbors` holds a node's edges to any other node.

```
1 typedef struct gnode {
2     Data data;
3     DataType type;
4     struct gnode *parent;    // tree mode only; NULL/unused for plain
                             // graphs
5     LLnode *neighbors;      // adjacency list: children (tree) or
                             // edges (graph)
6 } GNode;
```

6.1 GNodeArray

Traversal and search functions return results in a small resizable array rather than just printing as they go, so callers can use the results programmatically:

```
1 typedef struct {
2     GNode **items;
3     int count;
4     int capacity;
5 } GNodeArray;
```

`gnode_array_create`, `gnode_array_push`, and `gnode_array_free` manage it; `gnode_array_free` frees only the array wrapper, never the `GNodes` it points to, since the array does not own them.

6.2 The registry pattern

Both tree building and graph lookups need to find an existing node by its stored value. Rather than a hidden global (as in a typical classroom implementation), every function that needs a lookup table takes an explicit **registry** — a plain `LLnode` list of `TYPE_PTR` entries pointing at every node created so far:

```
1 void    register_node(LLnode **registry_head, GNode *node);
2 GNode *find_node(LLnode **registry_head, DataType type, Data key);
```


6.3 Tree mode

```
1 GNode *tree_add_node(LLnode **registry_head, GNode **root_ref,
2                      DataType type, Data parent_key, Data
                      child_value);
```

The first call (when `*root_ref == NULL`) creates the root. Every call after that looks the parent up in the registry by value and attaches a new child under it, returning `NULL` if the parent can't be found.

6.3.1 Traversals

Function	Order
<code>tree_preorder</code>	node, then each child's subtree (left to right)
<code>tree_postorder</code>	each child's subtree (left to right), then node
<code>tree_inorder</code>	<i>generalized N-ary inorder</i> : first child's subtree, then node, then the remaining children's subtrees

“Inorder” only has one obvious meaning for a *binary* tree (left, node, right). Since `GNode` allows any number of children, this project generalizes it the standard way described above — for an actual binary tree, that generalization collapses to exactly the usual left/node/right order.

6.4 Graph mode

```
1 void graph_add_edge(GNode *a, GNode *b, int directed);
2 // directed == 0 -> adds a->b AND b->a (undirected)
3 // directed == 1 -> adds only a->b (directed)
```

```
1 GNodeArray *graph_dfs(GNode *start);
2 GNodeArray *graph_bfs(GNode *start);
```

Both traversals guard against cycles with a *visited* list (itself just another `LLnode` list of `TYPE_PTR` entries) so a cyclic graph never causes infinite recursion or an infinite loop.

`graph_bfs` is built directly on top of the **Queue** described in Section 4 — `enqueue/dequeue` manage the BFS frontier exactly as they would for any other queue. This is the clearest example of the “unified” theme in this library: the tree, the graph, and the queue all share the same underlying `LLnode` plumbing.

For a graph with V reachable vertices and E edges: **DFS/BFS time:** $O(V + E)$ — amortized, since the visited-list membership check (`node_in_list`) is a linear scan, giving $O(V \cdot E)$ worst case for very dense graphs, though this is adequate for the small graphs this library targets (interview-prep / classroom scale).

7 ASCII-Art Visualization

Every structure has a dedicated print function that draws it the way it would be drawn by hand on paper, instead of a flat text dump.

7.1 Linked list, queue, stack: box diagrams

`ll_peek`, `display_queue`, and `display_stack` all render fixed-width boxes sized to their content, connected appropriately for each structure:

```

NULL <-> [ 5 ] <-> [ 10 ] <-> [ 20 ] <-> NULL      (ll_peek)

FRONT                                REAR
+----+ +----+ +----+
|  1  | |  2  | |  3  |
+----+ +----+ +----+                                (display_queue)

TOP
+----+
|  3  |
+----+
|  2  |
+----+                                (display_stack)
|  1  |
+----+
```

7.2 Tree: a recursive 2D layout

`print_tree` builds the diagram bottom-up. Each subtree is rendered into a small `TreeBlock` — a rectangular block of text lines, plus the column where that subtree’s own root sits (so a parent knows where to drop its branch line):

```

1 typedef struct {
2     char **lines;
3     int line_count;
4     int width;
5     int root_pos;
6 } TreeBlock;
```

`build_tree_block` recursively builds each child’s block, places them side by side with a small gap, then adds four rows above them: the parent’s label (centered over its children), a stem connecting the label down to a horizontal bar, the bar itself (spanning from the first child’s column to the last), and a row of stems dropping from the bar to each child:

```

      1
      |
+---+---+
|   |   |
2   3
```

7.3 Graph: circular layout with a hand-drawn line style

`print_graph` lays every node reachable from the start node evenly around an ellipse (computed with a self-contained Taylor-series `art_sin/art_cos`, so the build needs no `-lm`), then draws a line between any two nodes that share an edge using a Bresenham line-drawing algorithm on a character grid.

Early versions of the line drawer picked a different character per slope (|, -, /, \), which looked cluttered on steep diagonals (many slash characters packed together). The final version draws every line with a single . and adds one arrow character (>, <, ^, or v) right before the destination node, chosen from the line's dominant direction:

```

      (A)
      . \.....
      .      ....
(D) <.....      ....
      .              . (B)
      (C)
```

The arrow is placed on the *last dot before the destination*, not on the destination cell itself, and that placement explicitly skips over the column range the destination's own (`label`) box will occupy. Labels are drawn in a separate pass *after* every edge, so without this check a short, mostly-horizontal edge could have its arrowhead silently overwritten by the very label it was pointing at.

Every edge (directed or undirected) is drawn exactly once: a shared $n \times n$ `drawn[]` table is checked before drawing, regardless of which of the two endpoints the edge was stored under, so a directed edge whose source has a higher array index than its target is never missed.

8 Testing

`test.c` is a single self-contained test suite (no external test framework) covering every public function across all five structures.

8.1 Aligned, wrapped terminal logging

Rather than printing raw pass/fail lines, every stage of the test prints in a fixed-width (72-column), label-aligned format so it stays readable in a normal terminal window:

```
-----
STAGE : ll_add(&list, TYPE_INT, make_int(10), LL_END)
INPUT  : value=10, pos=LL_END
RESULT: list is now [10]
      [ PASS ] ll_add(LL_END) on empty list -> length 1
```

`print_wrapped` handles long lines by wrapping them onto further lines indented to line up under their own label, instead of letting them run off the edge of the screen.

8.2 Test structure

Function	Covers
<code>test_linked_list</code>	<code>ll_add</code> , both pops, <code>ll_del_at</code> , <code>ll_peek</code> , <code>ll_free_list</code> , strings
<code>test_sort</code>	<code>ll_sort</code> across int / string / float / char, plus empty & single-node
<code>test_queue</code>	<code>enqueue</code> , <code>dequeue</code> , <code>display_queue</code> , empty-queue safety
<code>test_stack</code>	<code>push</code> , <code>pop</code> , <code>display_stack</code> , empty-stack safety
<code>test_tree</code>	<code>tree_add_node</code> , <code>find_node</code> , <code>print_tree</code> , all three traversals
<code>test_graph</code>	<code>graph_add_edge</code> (directed & undirected), <code>graph_dfs</code> , <code>graph_bfs</code> , <code>print_graph</code>
<code>show_visual_gallery</code>	builds every structure fresh and calls its print function, with no test noise

The full suite runs **49 checks** across all six functional test functions, each reported with a `[ PASS ]` / `[ FAIL ]` tag, plus a final summary count. The process exit code is 0 only if every check passed, so the suite can be dropped into a CI pipeline directly.

9 Appendix: Full API Reference

Function	Description
Value builders	
<code>make_int / make_float / make_char</code>	wrap a primitive in a Data
<code>make_str(const char*)</code>	heap-copies a string into a Data
<code>make_ptr(void*)</code>	wraps a raw pointer in a Data
Linked list	
<code>create_node(type, data)</code>	allocate one LLnode
<code>ll_len(head_ref)</code>	length of the list
<code>ll_add(head_ref, type, value, pos)</code>	unified insert (<code>LL_END</code> = append)
<code>ll_pop_beg / ll_pop_end</code>	remove & return front/back value
<code>ll_del_at(head_ref, pos)</code>	delete without returning the value
<code>ll_peek(head_ref)</code>	draw as a box diagram
<code>ll_free_list(head_ref)</code>	free every node
<code>ll_sort(head_ref)</code>	ascending merge sort
Queue	
<code>enqueue(head_ref, type, value)</code>	insert at the back
<code>dequeue(head_ref, type_out)</code>	remove & return the front
<code>display_queue(head_ref)</code>	draw with FRONT/REAR labels
Stack	
<code>push(head_ref, type, value)</code>	insert at the top
<code>pop(head_ref, type_out)</code>	remove & return the top
<code>display_stack(head_ref)</code>	draw with a TOP label
Tree / Graph node	
<code>create_gnode(type, value)</code>	allocate one GNode
<code>print_gnode_value(node)</code>	print one node's value
<code>gnode_array_create / push / free</code>	manage a GNodeArray
<code>print_gnode_array(arr)</code>	print an array as [a, b, c]
<code>register_node(registry, node)</code>	add a node to a lookup registry
<code>find_node(registry, type, key)</code>	look a node up by its value
Tree mode	
<code>tree_add_node(registry, root, ...)</code>	create root / attach a new child
<code>print_tree(node, level)</code>	draw the full 2D branching diagram
<code>tree_preorder / inorder / postorder</code>	return a GNodeArray in that order
Graph mode	
<code>graph_add_edge(a, b, directed)</code>	add an edge (0 = undirected)
<code>graph_dfs(start)</code>	depth-first traversal, cycle-safe
<code>graph_bfs(start)</code>	breadth-first traversal (reuses the Queue)
<code>print_graph(start)</code>	draw nodes on a circle with dotted, arrowed edges

10 Appendix: Complexity Summary

Operation	Time	Notes
<code>ll_len</code>	$O(n)$	walks the list
<code>ll_add</code> (front / end)	$O(1)$ / $O(n)$	end requires walking to find n first
<code>ll_add</code> (arbitrary pos)	$O(n)$	walk to the position
<code>ll_pop_beg</code>	$O(1)$	
<code>ll_pop_end</code>	$O(n)$	walks to the last node
<code>ll_sort</code>	$O(n \log n)$	merge sort
<code>enqueue</code> / <code>dequeue</code>	$O(1)$	back-linked list, no re-walk
<code>push</code> / <code>pop</code> (stack)	$O(1)$	same reasoning
<code>find_node</code>	$O(r)$	r = number of registered nodes
<code>tree_add_node</code>	$O(r)$	dominated by the <code>find_node</code> lookup
<code>tree_preorder</code> / <code>in</code> / <code>post</code>	$O(k)$	k = number of nodes in the tree
<code>graph_dfs</code> / <code>graph_bfs</code>	$O(V + E)^*$	V, E = reachable vertices/edges
<code>print_graph</code>	$O(V + E)^*$	plus $O(V \cdot W \cdot H)$ for the grid draw

* amortized; visited-set membership is a linear scan, so dense graphs approach $O(V \cdot E)$ worst case.